

SIH26166 — Team Work Division & Implementation Plan

Pre-implementation team handoff based on the Master Engineering & Research Specification v3.0.

1. Team ownership

Person	Ownership	Dependency / start
Tejas	Architecture, integration, pipeline, data contracts	Day 1; everyone depends on interfaces
Haruto	PDS/data ingestion, preprocessing, manifests	Day 1; depends mainly on Tejas's LunarProduct contract
Shashwat	SPICE, geometry, pair characterization	Day 1; uses Tejas's product/pair contracts
Chuba	Representations, matching, adaptive routing	Day 1 after interfaces; consumes Haruto products and later Shashwat
Shaiz	Verification, control points, sub-pixel, registration, evaluation/UI	After correspondence interface; consumes Haruto + Chuba outputs

2. Tejas — Architecture + Integration Lead

- Own: docs/, src/models/, src/pipeline/, src/io/, configs/ and repository standards.
- Define the canonical interfaces: LunarProduct, RegistrationPair, CorrespondenceSet and RegistrationResult.
- Own the end-to-end orchestration: ingest → characterize → preprocess → representation → match → verify → control points → refine → register → evaluate → export.
- Maintain configuration, integration tests, architecture consistency, decision log and final integration.
- Do not try to code every module. Ensure that independent modules connect correctly.

3. Haruto — Data + PDS + Preprocessing

- Own: src/ingestion/, src/preprocessing/, data/manifests/.
- Build product readers, metadata extraction, validation, masks, provenance and the canonical LunarProduct.
- Capture instrument, mission, dimensions, GSD/resolution, acquisition time, radiometric state, coordinates, valid pixels and provenance where available.
- Initial flow: load → validate → mask → normalize → orient/resample when required.
- First milestone: one real lunar image + mask + metadata successfully represented as LunarProduct.

4. Shashwat — SPICE + Geometry + Pair Characterization

- Own: src/geometry/ and pair-characterization logic.
- Create a centralized SPICE/geometry interface rather than scattering SPICE calls through the codebase.
- Provide spacecraft state, camera geometry and Sun geometry where available.
- Compute pair characteristics such as sensor pair, scale ratio, Sun-angle difference, viewing difference, overlap, quality and difficulty.
- Eventually provide the inputs used by adaptive routing: easy / normal / difficult.
- Start with mock/sample metadata if necessary, then connect real scientific products.

5. Chuba — Matching + Representation + Adaptive Routing

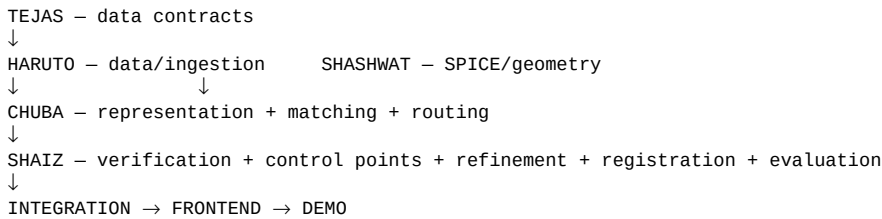
- Own: src/representation/, src/matching/, src/routing/ and matcher experiments.
- Start with a small shortlist: SIFT/ASIFT baseline, RIFT/RIFT2 and LightGlue; later evaluate one difficult/dense candidate such as LoFTR-family and optionally a specialized illumination method such as PWIFT.
- Common interface: match(source, reference, metadata) → CorrespondenceSet.
- Build intensity/gradient/structural representations first; add phase and IIRS-specific representations through experiments.
- Run the controlled matcher benchmark and select the best measured accuracy/robustness/runtime trade-off.

6. Shaiz — Verification + Registration + Refinement + Evaluation/UI

- Own: verification, control_points, refinement, registration, evaluation and frontend.
- Stage 1: CorrespondenceSet → filtering → robust verification → inliers → residuals.
- Stage 2: grid/quadtrees → candidates → quality ranking → spatial diversity → final control points.
- Stage 3: estimate the appropriate transformation/model and create the registered source image.

- Stage 4: coarse points → local refinement → sub-pixel coordinates → uncertainty → final refit.
- Stage 5: RMSE, MAE, median error, 95th percentile, inlier count/ratio, coverage, uniformity and runtime.
- Build the polished UI after the scientific pipeline works.

7. Dependency graph



8. First sprint

Tejas: define the four core data contracts and initialize repository structure.

Haruto: load one real lunar product and output image + mask + metadata.

Shashwat: create geometry interface and begin SPICE integration.

Chuba: implement SIFT through the common matcher interface.

Shaiz: implement correspondence → robust verification → baseline transformation → registered image → basic metrics.

9. First real milestone

REAL LUNAR DATA → LunarProduct → SIFT matches → robust verification → registration → registered image → metrics

Progress is measured by reaching a real end-to-end scientific baseline, not by lines of code or number of models.

10. Collaboration rules

- One person → one feature/module → one branch → one pull request.
- Keep main stable; do not develop directly on main.
- Agree interfaces before large implementations.
- Avoid giant shared scripts containing the entire pipeline.
- Accuracy-affecting scientific changes require benchmarks/validation.
- Architecture changes go into the decision log.
- Do not commit raw scientific datasets or credentials unless explicitly permitted.

11. Do not do initially

- Do not implement every candidate model before the baseline.
- Do not build the full frontend before registration works.
- Do not build an oversized SPICE framework before identifying required products.
- Do not add infrastructure complexity before the scientific pipeline works.
- Do not claim a matcher is globally best; benchmark it on the defined lunar data.

12. Execution sequence

Day 1: interfaces + repository + first data reader + geometry interface + SIFT baseline + baseline registration.

Then: end-to-end MVP → controlled matcher benchmark → illumination/scale/multimodal improvements → geometry/SPICE → spatial control points → sub-pixel refinement → adaptive routing → demo polish → official SIH dataset integration.